



IdentityWallet

STUDY NOTES

Understand your project, end to end

Every concept behind the Digital Identity Wallet explained simply, with analogies, worked examples, a code map and a one-page cheat sheet. Includes short Hinglish summaries.

Concepts

Cryptography

Flows

Code map

Cheat sheet

Student **Cecil Baraik · 2025-BSCIT/004A**
Programme **B.Sc. Information Technology**
Institution **Amity University Jharkhand**

Contents

1 The Big Picture

- 1.1 A story to remember
- 1.2 The three roles (the "trust triangle")
- 1.3 What the project gives (8 features)

2 Cryptography Basics (the building blocks)

- 2.1 Hashing — a fingerprint of data
- 2.2 Salt — stopping guessing attacks
- 2.3 Symmetric encryption — AES-256-GCM
- 2.4 Asymmetric cryptography — ECDSA P-256
- 2.5 Which algorithm where?

3 Selective Disclosure — Step by Step

- 3.1 Worked example
- 3.2 Why it can't be cheated
- 3.3 Derived claims

4 Identity, Trust and Keys

- 4.1 DIDs
- 4.2 Trust registry
- 4.3 Envelope encryption

5 Sharing and Verification Internals

- 5.1 What is inside a share link
- 5.2 The presentation
- 5.3 The five checks
- 5.4 Share status order

6 Account Security

- 6.1 Password storage (PBKDF2)
- 6.2 Login with 2FA
- 6.3 TOTP in one line
- 6.4 JWT structure
- 6.5 Brute-force and abuse protection
- 6.6 Access control

7 The Tamper-evident Audit Log

8 Spring Boot — How the Backend Works

8.1 Layers

8.2 Key Spring concepts

8.3 Trace one request: creating a share

9 React — How the Frontend Works

10 Code Map — Where Is What?

11 Glossary

12 One-page Cheat Sheet

How to read these notes

Read chapters 1–3 first: they explain the core idea. Chapters 4–7 explain the security features. Chapters 8–10 connect everything to the actual code. Chapter 12 is a one-page revision sheet for the night before the viva. Pink boxes give the same idea in simple Hinglish.

1 The Big Picture

1.1 A story to remember

Aarav wants to watch an 18+ film. Today he shows his ID card and the cinema staff see his **name, date of birth, address and ID number** — they only needed to know he is over 18.

With IdentityWallet:

1. The **Government office (issuer)** has already put a digitally signed *National Identity Card* into Aarav's wallet.
2. At the cinema, Aarav opens the card, chooses **"Age check only"**, types "City Cinema", picks *1 day, 3 views*, and shows the **QR code**.
3. The cinema (verifier) scans it. Their screen says **Verified — Age over 18: Yes**, and "7 other details were kept private".
4. Later Aarav sees in his **access log** that City Cinema opened it once, at 6:42 pm, from an Android phone. He can **stop sharing** any time.

Simple mein

Pehle hum poora ID card dikhate the, ab sirf utna dikhate hain jitna zaroori hai — "18+ hai ya nahi". Aur yeh jhoot nahi ho sakta kyunki government ka digital signature laga hai.

1.2 The three roles (the "trust triangle")

Role	Real-world example	Key it uses	What it does
Issuer	Govt. office, college, RTO	Issuer private key	Signs credentials
Holder	Aarav	Holder private key + data key	Stores, chooses, consents
Verifier	Cinema, bank, employer	Issuer and holder public keys	Checks signatures

The verifier never needs to call the issuer. It trusts the issuer's **public key** in the trust registry — like a shopkeeper who recognises a bank's stamp without phoning the bank.

1.3 What the project gives (8 features)

Signed credentials — issued by trusted organisations, impossible to forge.

Selective disclosure — share chosen details only, even "age over 18".

Consent controls — recipient, purpose, expiry, view limit, stop sharing.

Access log — who opened a share, when, from which device.

Public verifier — link, QR scan or JSON upload, no account.

Encrypted vault — passport scans etc. with AES-256-GCM.

Account security — PBKDF2, 2FA (TOTP),
lockout, rate limits.

Tamper-evident audit log — hash-chained events.

2 Cryptography Basics (the building blocks)

2.1 Hashing — a fingerprint of data

A **hash function** turns any input into a fixed-size "fingerprint". SHA-256 always gives 256 bits (64 hex characters).

```
SHA-256("Aarav") = 3f1c...9b2e
SHA-256("Aarav.") = a74d...01c7 ← one dot changes everything (avalanche effect)
```

Properties: **one-way** (can't get the input back), **deterministic** (same input → same hash), **collision-resistant** (can't find two inputs with the same hash).

Used in the project for: selective-disclosure digests, file integrity in the vault, storing share tokens, the audit chain, DIDs.

Simple mein

Hash ek fingerprint jaisa hai. Data se fingerprint ban jaata hai, par fingerprint se data wapas nahi ban sakta. Ek letter bhi badla toh fingerprint poora badal jaata hai.

2.2 Salt — stopping guessing attacks

If we hash only a birth date, someone can hash all ~22,000 dates between 1950 and 2010 and find the match in milliseconds. A **salt** is a random value added before hashing:

```
SHA-256("k3Fq9xPa..." + "2005-03-14") ← attacker would need to guess the 128-bit salt too
```

Salts are not secret — they are just unpredictable. Used for passwords (PBKDF2) and for every disclosure.

2.3 Symmetric encryption — AES-256-GCM

One key locks and unlocks. Very fast, so it is used for bulk data.

- **AES-256:** the cipher, with a 256-bit key.
- **GCM mode:** also produces an **authentication tag**. If even one bit of the ciphertext is changed, decryption *fails* instead of giving garbage — so it gives **confidentiality + integrity**.
- **IV (nonce):** 12 random bytes, new for every encryption, stored in front of the ciphertext.
- **AAD:** extra context (e.g. `vault-file:42`) that must match on decryption, so encrypted data can't be moved to another row.

```
stored blob = [ 12-byte IV ][ ciphertext ][ 16-byte tag ]
```

Simple mein

Ek hi chaabi se taala lagta aur khulta hai. GCM ek "seal" bhi lagata hai — agar kisi ne dabba chheda toh khulega hi nahi.

2.4 Asymmetric cryptography — ECDSA P-256

A **key pair**: a **private key** (secret) and a **public key** (shared with everyone).

- **Sign** with the private key → anyone **verifies** with the public key.
- ECDSA on the **P-256** curve gives security like RSA-3072 with much smaller keys (256 bits).

A **digital signature** proves:

Property	Meaning
Authenticity	It was signed by the owner of that private key
Integrity	The data hasn't changed since signing
Non-repudiation	The signer can't deny it later

Simple mein

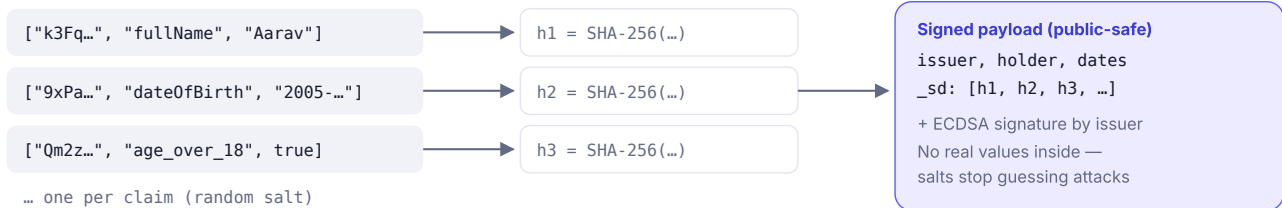
Private key = aapka asli sign/stamp, sirf aapke paas. Public key = sabke paas ek copy jisse woh check kar sakein ki stamp asli hai. Stamp ki nakal nahi ban sakti.

2.5 Which algorithm where?

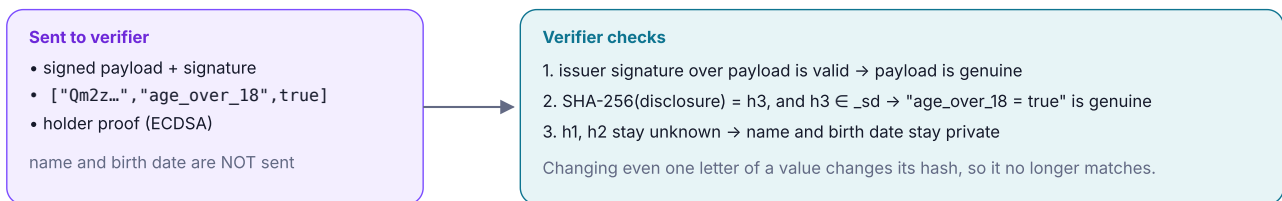
Algorithm	Type	Where in the project
ECDSA P-256 + SHA-256	Signature	Issuer signs credentials; holder signs share proofs
AES-256-GCM	Symmetric encryption	Vault, claims, private keys, data keys, TOTP secrets, shares
SHA-256	Hash	Disclosure digests, file integrity, token hash, audit chain, DID
PBKDF2-HMAC-SHA256 (210,000 rounds)	Slow password hash	User passwords
HMAC-SHA256	Keyed hash	JWT session signature
HMAC-SHA1 (TOTP)	One-time codes	Two-factor authentication
SecureRandom	Randomness	Salts, IVs, tokens, keys

3 Selective Disclosure — Step by Step

A. Issuance — each claim becomes a salted disclosure; only its hash is signed



B. Presentation — holder reveals only what is needed



Only salted hashes are signed; the holder reveals a subset

3.1 Worked example

Step 1 — Issuer makes a disclosure for every claim.

```
d1 = base64url( ["k3Fq...", "fullName", "Aarav Sharma"] )
d2 = base64url( ["9xPa...", "dateOfBirth", "2005-03-14"] )
d3 = base64url( ["Qm2z...", "age_over_18", true] )
```

Step 2 — Hash each disclosure. $h1 = \text{SHA-256}(d1)$, $h2 = \text{SHA-256}(d2)$, $h3 = \text{SHA-256}(d3)$.

Step 3 — Sign a payload that contains only the hashes.

```
{ "id": "cred_...", "issuer": { "did": "did:idw:...", "name": "National Identity Authority" },
  "holder": "did:idw:...", "_sd_alg": "sha-256", "_sd": [ "h1", "h2", "h3" ] }
signature = ECDSA_sign(issuerPrivateKey, payload)
```

Step 4 — Wallet stores payload + signature (public-safe) and d1, d2, d3 **encrypted** with Aarav's data key.

Step 5 — Aarav shares only d3 (age over 18), plus the payload and signature and his holder proof.

Step 6 — Cinema verifies: signature valid ✓ → $\text{SHA-256}(d3)$ equals h3 which is in $_sd$ ✓ → shows "Age over 18: Yes". h1 and h2 reveal nothing.

3.2 Why it can't be cheated

Attack	Why it fails
Change "false" to "true" in d3	New hash $\neq$ h3, not in $_sd$ → "Revealed details" ×

Attack	Why it fails
Add a fake claim with a new hash	Adding to <code>_sd</code> breaks the issuer signature → "Issuer signature" ×
Guess the hidden birth date from h2	Needs the 128-bit random salt → infeasible
Reuse someone else's presentation	Holder proof is bound to the original recipient and time → "Holder consent" ×

Simple mein

Issuer har detail ka "taala-band fingerprint" sign karta hai, detail khud nahi. Aarav jo detail dikhana chahe uski chaabi (salt + value) de deta hai. Cinema fingerprint match karta hai. Baaki details ke fingerprints se kuch pata nahi chalta.

3.3 Derived claims

The issuer computes `age_over_18` and `age_over_21` from the date of birth **at issuance** (`CredentialService.addDerivedClaims`). They are signed like normal claims, so Aarav proves his age without revealing the date. This is **data minimisation**.

4 Identity, Trust and Keys

4.1 DID

Every user and issuer gets a key pair when created, and a **Decentralized Identifier**:

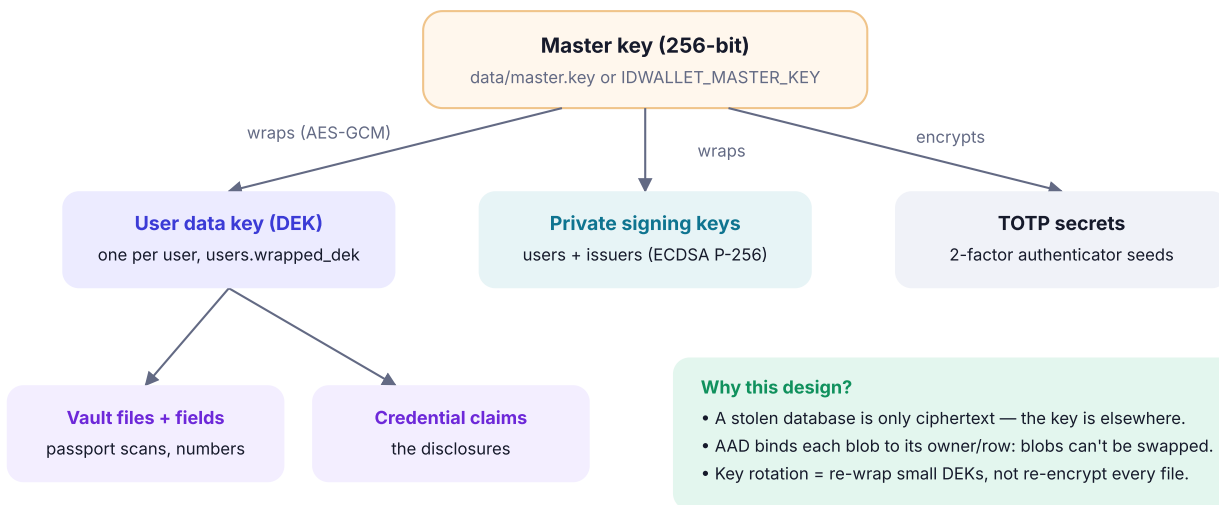
```
did:idw:<first 20 bytes of SHA-256(public key), in hex>
e.g. did:idw:df3d86746f9d0116315d16de12fec3a3eb628d6c
```

The DID is tied to the key: anyone with the public key can recompute it. The **key fingerprint** shown in the UI (e.g. `DF3D:8674:6F9D:0116`) is the first 8 bytes of the same hash, easy for humans to compare.

4.2 Trust registry

The `issuers` table is the list of organisations whose signatures are accepted. Only the **admin** can add issuers. If a hacker generates their own key and signs a fake credential, the verifier finds the issuer DID is **not in the registry** → "Trusted issuer" ×. Suspending an issuer makes all its credentials fail immediately.

4.3 Envelope encryption



One master key protects everything; it is never in the database

Layer	Key	Protects
1	Master key (file <code>data/master.key</code> or env var)	Wraps everything below
2	Data key (DEK) per user, stored wrapped in <code>users.wrapped_dek</code>	That user's vault and credential claims
2	Private keys of users and issuers, stored wrapped	Signing
2	TOTP secrets , stored encrypted	Two-factor codes

Simple mein

Ek "master chaabi" hai jo database ke bahar rakhi hai. Har user ki apni chaabi hai, jo master chaabi se band hai. Database chori bhi ho jaye toh sirf band dabbe milenge.

5 Sharing and Verification Internals

5.1 What is inside a share link

`http://localhost:5173/v/lpzpd2IS_P95yXxk17r_8gRhecBtXczX` — the last part is a **random 192-bit token** (24 bytes, Base64url).

Stored in DB	Not stored
SHA-256(token)	The token itself
Presentation encrypted with <code>SHA-256("idwallet-share-key " + token)</code>	The presentation in plain text
Token encrypted with the holder's data key (to show the QR again)	—

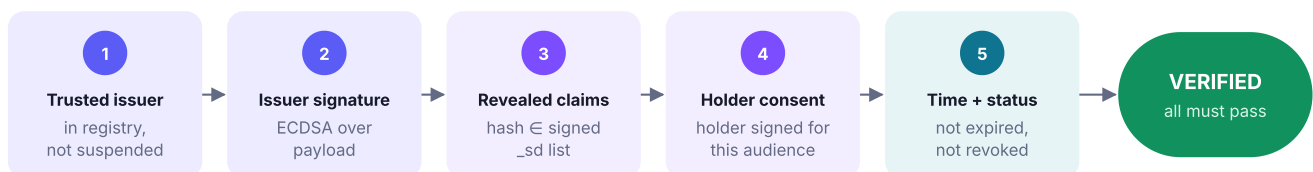
So the database alone can neither open a link nor read the shared data.

5.2 The presentation

```
{ "format": "idwallet-vp+sd",  
  "credential": "<signed payload>", "issuerSignature": "<ECDSA>",  
  "disclosures": [ "<only the chosen ones>" ],  
  "holderProof": { "holder": "did:idw:...", "audience": "City Cinema", "nonce": "...",  
                   "createdAt": "...", "expiresAt": "...", "signature": "<holder ECDSA>" } }
```

The **holder proof** signs `SHA-256(payload) | disclosures | audience | nonce | createdAt | expiresAt` with Aarav's private key. That proves **Aarav consented** to show *these* details to *City Cinema* until *that time*.

5.3 The five checks



All five must pass for "Verified"

5.4 Share status order

When a link is opened the server checks, in order: **revoked** → **expired** → **used up (views ≥ max)** → **active**. Only an active share is decrypted, verified and its view counter increased. Every attempt is written to `access_logs`.

6 Account Security

6.1 Password storage (PBKDF2)

```
stored = "pbkdf2$210000$<salt base64>$<hash base64>"
hash   = PBKDF2-HMAC-SHA256(password, 16-byte random salt, 210,000 iterations, 256 bits)
```

210,000 iterations $\approx$ 0.4 s per guess, so stolen hashes are very slow to crack. Comparison uses `MessageDigest.isEqual` (constant time).

6.2 Login with 2FA

- 1 Email + password correct $\rightarrow$ if 2FA is on, server returns an **MFA ticket** (JWT, kind = "mfa", 5 minutes) — not a session.
- 2 Ticket + 6-digit code $\rightarrow$ server checks TOTP $\rightarrow$ returns an **access token** (JWT, kind = "access", 8 hours).
- 3 Every API call sends `Authorization: Bearer <token>`.

6.3 TOTP in one line

```
code = last 6 digits of truncate( HMAC-SHA1(secret, floor(unixTime / 30)) )
```

Phone and server compute the same code because they share the secret (via the QR code) and the time. One window (± 30 s) on each side is accepted for clock drift.

6.4 JWT structure

```
base64url({"alg":"HS256","typ":"JWT"}) . base64url({"sub":"5","email":"...","role":"USER",
"kind":"access","iat":..., "exp":..., "jti":"..."}) . base64url(HMAC-SHA256(secret, header.payload))
```

Change one character $\rightarrow$ signature mismatch $\rightarrow$ 401.

6.5 Brute-force and abuse protection

Protection	Rule
Account lockout	5 wrong passwords $\rightarrow$ locked 5 minutes
Login rate limit	20 per IP per 5 minutes
MFA rate limit	10 per IP per 5 minutes
Sign-up rate limit	5 per IP per hour
Verification rate limit	60 per IP per minute

Protection	Rule
Same error message	"Wrong email or password" for both cases
Dummy hash	Unknown emails take the same time as real ones

6.6 Access control

- **401** — no or invalid token (`AuthUser` resolver).
- **403** — wrong role (`AuthUser.require(Role.ISSUER)`).
- **404** — record exists but belongs to someone else (IDOR protection).

7 The Tamper-evident Audit Log

```
event 1: prevHash = 000...0      hash = SHA-256(000...0 | data1) = A
event 2: prevHash = A            hash = SHA-256(A | data2)   = B
event 3: prevHash = B            hash = SHA-256(B | data3)   = C
```

If someone edits event 2 in the database, its recomputed hash is no longer B, and event 3's `prevHash` no longer matches. **Verify hash chain** walks all events and reports the first broken one.

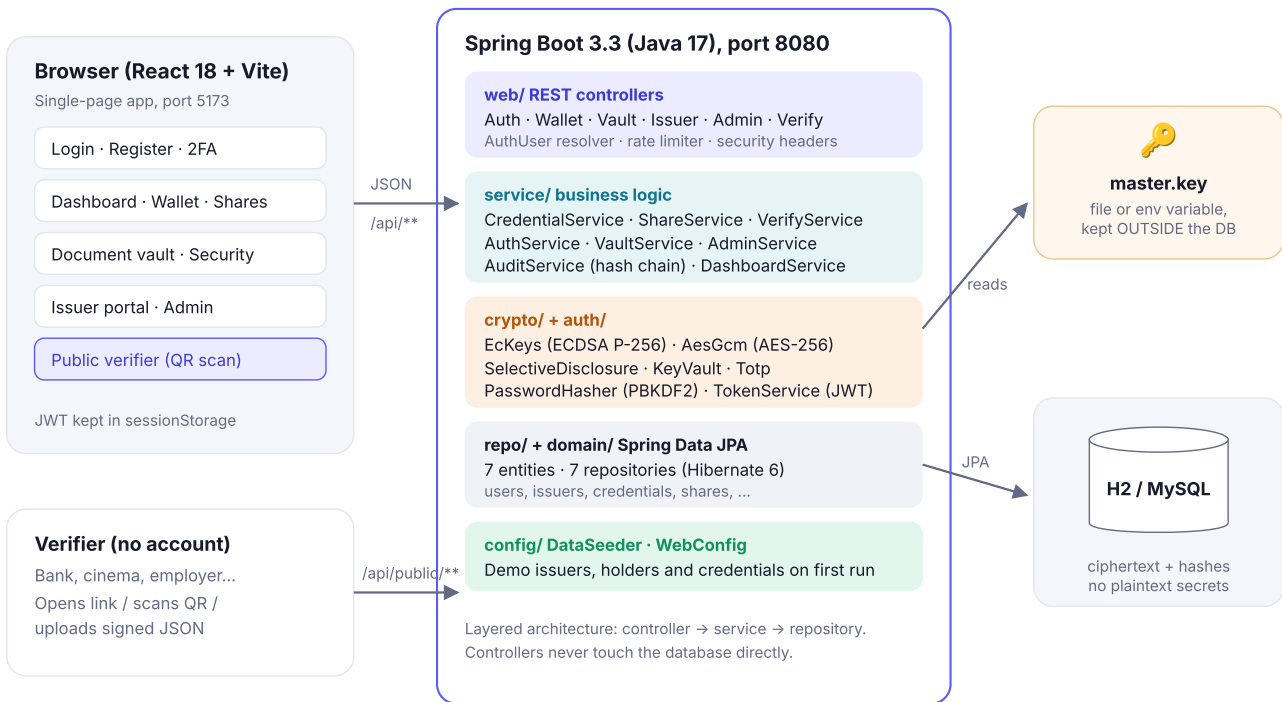
Simple mein

Har entry pichhli entry ke fingerprint se judi hai, jaise chain ki kadiyan. Ek kadi chhedi toh aage ki saari kadiyan toot jaati hain — turant pakda jaata hai.

Events logged include: registration, login, failed login, MFA on/off, password change, credential issued/received/revoked/removed, share created/viewed/revoked, vault add/update/download/delete, issuer registered/suspended, user enabled/disabled.

8 Spring Boot — How the Backend Works

8.1 Layers



Architecture

Layer	Package	Job	Example
Controller	web	Receive HTTP, call a service, return JSON	<code>WalletController.share()</code>
Service	service	Business rules, transactions	<code>ShareService.create()</code>
Repository	repo	Database queries	<code>ShareRepository.findByTokenHash()</code>
Entity	domain	Table mapping	Share → table shares
Crypto / auth	crypto, auth	Security primitives	<code>EcKeys.sign()</code>

8.2 Key Spring concepts

Concept	One-line meaning	In the project
<code>@SpringBootApplication</code>	Starts the app and scans components	<code>IdWalletApplication</code>
<code>@RestController</code>	Class whose methods return JSON	All files in <code>web/</code>
<code>@Service</code> , <code>@Component</code>	Beans created and injected by Spring	Services, <code>KeyVault</code> , <code>RateLimiter</code>

Concept	One-line meaning	In the project
Constructor injection	Dependencies passed into the constructor	Every service
<code>@Entity</code> , <code>@Id</code> , <code>@Column</code>	Map a class to a table	<code>domain/*</code>
<code>JpaRepository</code>	Ready-made CRUD + derived queries	<code>repo/*</code>
<code>@Transactional</code>	All-or-nothing database work	Service methods
<code>@Value("\${...}")</code>	Read <code>application.properties</code>	Master key, JWT secret
<code>@RestControllerAdvice</code>	Global error handling	<code>GlobalExceptionHandler</code>
<code>ApplicationRunner</code>	Code that runs at startup	<code>DataSeeder</code> (demo data)
Profiles	Switch config	<code>mysql</code> profile

8.3 Trace one request: creating a share

- 1 React calls `POST /api/wallet/shares` with JSON and the Bearer token.
- 2 `WebConfig`'s resolver validates the JWT and creates an `AuthUser`.
- 3 `WalletController.share()` passes the request to `ShareService.create()`.
- 4 The service checks ownership and status, decrypts the disclosures with the holder's data key (`KeyVault` → `AesGcm`), keeps only the chosen ones.
- 5 It creates the token, signs the holder proof (`EcKeys.sign`), encrypts the presentation, saves the `Share` via `ShareRepository`.
- 6 `AuditService.log()` appends a hash-chained event.
- 7 The controller returns JSON with the link; React shows the QR code.

9 React — How the Frontend Works

File	Role
<code>main.jsx</code>	Starts React with Router, Toast and Auth providers
<code>App.jsx</code>	Routes and role guards (holder / issuer / admin)
<code>auth.jsx</code>	Login state (context); restores session on refresh
<code>api.js</code>	<code>fetch</code> wrapper: adds the token, handles errors and 401
<code>components/Layout.jsx</code>	Sidebar + top bar, menu by role
<code>components/ShareLink.jsx</code>	QR code + copy/open link
<code>pages/*.jsx</code>	One file per screen

React ideas used: components, props, `useState` (form values, dialogs), `useEffect` (load data when a page opens), context (logged-in user), React Router (`/wallet/:id` , `/v/:token`).

10 Code Map — Where Is What?

If the examiner asks about...	Open this file
AES-GCM encryption	<code>crypto/AesGcm.java</code>
ECDSA keys, signing, DID	<code>crypto/EcKeys.java</code>
Master key / envelope encryption	<code>crypto/KeyVault.java</code>
Selective disclosure	<code>crypto/SelectiveDisclosure.java</code>
TOTP	<code>crypto/Totp.java</code>
Password hashing	<code>auth/PasswordHasher.java</code>
JWT	<code>auth/TokenService.java</code>
Rate limiting	<code>auth/RateLimiter.java</code>
Login, logout, 2FA flow	<code>service/AuthService.java</code>
Issuing, derived claims, revocation	<code>service/CredentialService.java</code>
Credential templates	<code>service/CredentialTemplates.java</code>
Shares, holder proof	<code>service/ShareService.java</code>
Verification checks	<code>service/VerifyService.java</code>
Vault, magic-byte check	<code>service/VaultService.java</code>
Audit hash chain	<code>service/AuditService.java</code>
Security score	<code>service/DashboardService.java</code>
Security headers, auth resolver	<code>config/WebConfig.java</code>
Demo data	<code>config/DataSeeder.java</code>
Settings	<code>resources/application.properties</code>
Tests	<code>test/.../CryptoTest.java</code> , <code>WalletFlowTest.java</code>

11 Glossary

Term	Meaning
AAD	Additional Authenticated Data — context checked by AES-GCM but not encrypted
Claim	One fact in a credential, e.g. <code>dateOfBirth = 2005-03-14</code>
Credential	A set of claims signed by an issuer
DEK	Data Encryption Key — a user's own AES key
DID	Decentralized Identifier, derived from a public key
Digest	Output of a hash function
Disclosure	<code>[salt, name, value]</code> — opens one digest
ECDSA	Elliptic Curve Digital Signature Algorithm
Envelope encryption	Keys encrypting keys encrypting data
GCM	Galois/Counter Mode — authenticated AES mode
Holder proof	Holder's signature binding a share to a recipient and time
IDOR	Accessing someone else's record by changing an ID
IV / nonce	Unique value per encryption / per proof
JWT	Signed token carrying the session
Master key	Top-level key stored outside the database
PBKDF2	Slow, salted password hashing
Presentation	What the holder sends to a verifier
Revocation	Issuer withdrawing a credential
Salt	Random value added before hashing
Selective disclosure	Revealing only chosen claims
TOTP	Time-based One-Time Password (6 digits / 30 s)
Trust registry	List of issuers whose keys are accepted
Verifier	Party checking a presentation

12 One-page Cheat Sheet

Stack

Java 17 · Spring Boot 3.3 · JPA/Hibernate · H2 / MySQL · React 18 · Vite · JUnit 5

Roles

Holder · Issuer · Admin · Verifier (no account)

Algorithms

ECDSA P-256 (signatures) · AES-256-GCM (encryption) · SHA-256 (hashes) · PBKDF2 210k (passwords) · HMAC-SHA256 (JWT) · TOTP RFC 6238 (2FA)

5 verification checks

1 Trusted issuer · 2 Issuer signature · 3 Revealed details · 4 Holder consent · 5 Time + status

Numbers

12-byte IV · 128-bit GCM tag · 128-bit disclosure salt · 192-bit share token · 256-bit keys · 30 s TOTP · 5 tries → 5 min lock · 5 MB upload · 8 h session

Tests

8 tests, all pass: 5 crypto unit + 3 integration (forgery, view limit, revocation, 401/403/404, fake PDF, MFA, lockout, audit chain)

Privacy principles

Data minimisation · consent · purpose · expiry · transparency (access log) · encryption at rest

Future scope

Mobile wallet + secure enclave · passkeys · SD-JWT VC / W3C VC 2.0 · status lists · ZKP (BBS+) · Docker + HTTPS

Ek line mein poora project

Issuer sign karta hai, Holder encrypted rakhta hai aur sirf zaroori detail share karta hai, Verifier 5 checks se turant confirm karta hai — aur har cheez ka log banta hai.